

Lean

PENG

Created: 20260910, Version: 20260910

Contents

1	Introduction	1
1.1	Evaluating Expressions	1
1.2	Types	1
1.3	Functions and Definitions	1
1.3.1	Defining Functions	2
1.3.2	Defining Types	2
1.4	Structures	2

1 Introduction

Lean is a **expression-oriented** functional language with dependent types [1].

1.1 Evaluating Expressions

- every expression must have a type before it can be evaluated
- once given a value, variables cannot be reassigned
- parentheses are necessary when a function's argument is to be computed via another function call, i.e. $5 * (1 + 2)$

1.2 Types

Definition 1.1 (type).

- `Nat`: *default type for non-negative integer literals*
- `Int`
- `Float`
- `String`
- `Bool`

Example 1.2.

```
#eval (1 - 2 : Nat) -- 0
#check (1 - 2 : Int) -- check the type of an expression
```

1.3 Functions and Definitions

Definition 1.3 (definition).

```
def var : type := val
```

Example 1.4.

```
def lean : String := "Lean"
#eval String.append "hello " lean
```

1.3.1 Defining Functions

Definition 1.5 (function).

```
def func (arg1 : type) ... (argn : type) : type := body
```

Example 1.6.

```
def maximum (n : Nat) (m : Nat) : Nat := if n < m then m else n
#eval maximum 1 5
```

Definition 1.7 (currying). $\text{Nat} \rightarrow (\text{Nat} \rightarrow \text{Nat})$

1.3.2 Defining Types

Definition 1.8 (Type).

```
def Str : Type := String -- overloaded
abbrev N : Type := Nat -- unfolded
```

Definition 1.9 (overloaded).

Definition 1.10 (reducible). *Definitions that are to be unfolded are called reducible.*

1.4 Structures

Definition 1.11 (structure).

```
structure Point where
x : Float
y : Float
def origin : Point := {x:=0.0,y:=0.0}
#eval origin.x
#check {x:=0.0,y:=0.0:Point}
```

Definition 1.12 (updating structures). `def zeroX (p : Point) := {p with x:=0}`

Definition 1.13 (constructor).

```
structure Point where
point::
x : Float
y : Float
#check Point.point 1.0 2.0
```

Definition 1.14 (accessor).

```
#eval Point.x {x:=1.0,y:=2.0}

-- TARGET.func ARG1 ... ARGn
#eval "hello".append " Lean"
def Point.modifyBoth (f : Float -> Float) (p : Point) : Point := {x:=f p.x,y:=f p.y}
#eval {x:=3,y:=4 : Point}.modifyBoth Float.floor
```

References

- [1] David Thrane Christiansen. *Functional programming in Lean*. 2023.